Практическая работа №2. Анализ протокола HTTP.

Задание №1. Загрузка файла трассировки http.

Файл трассировки trace-http.pcap был открыт в программе Wireshark. С помощью меню Статистика - Свойства файла захвата были получены следующие данные:

1. Общее количество перехваченных пакетов: 1398
2. Общий объем перехваченных пакетов: 1 223 530 байт

Для подсчета HTTP-пакетов были применены фильтры отображения:

1. Фильтр http.request показал 98 пакетов HTTP-запросов.
2. Фильтр http.response показал 98 пакетов HTTP-ответов.

Равное количество запросов и ответов объясняется тем, что на каждый HTTP-запрос от клиента сервер возвращает один ответ.

[Здесь вставь 3 скриншота: общий вид файла, свойства с фильтром http.request, свойства с фильтром http.response]

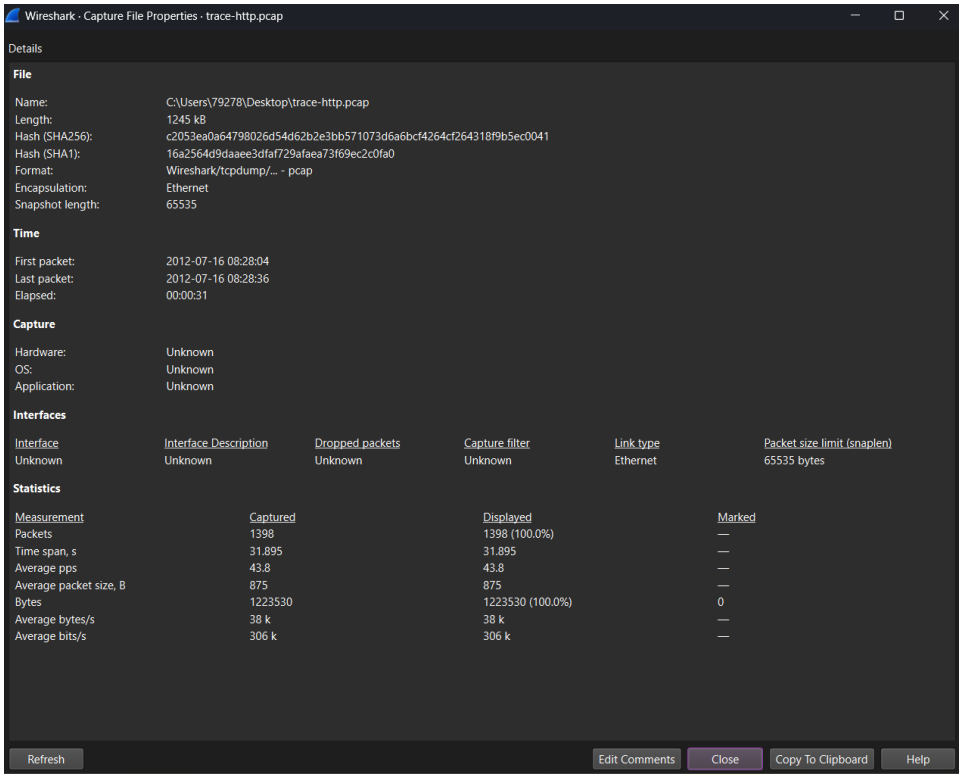


Рисунок 1 — Свойства файла захвата trace-http.pcap (общая статистика без фильтров)

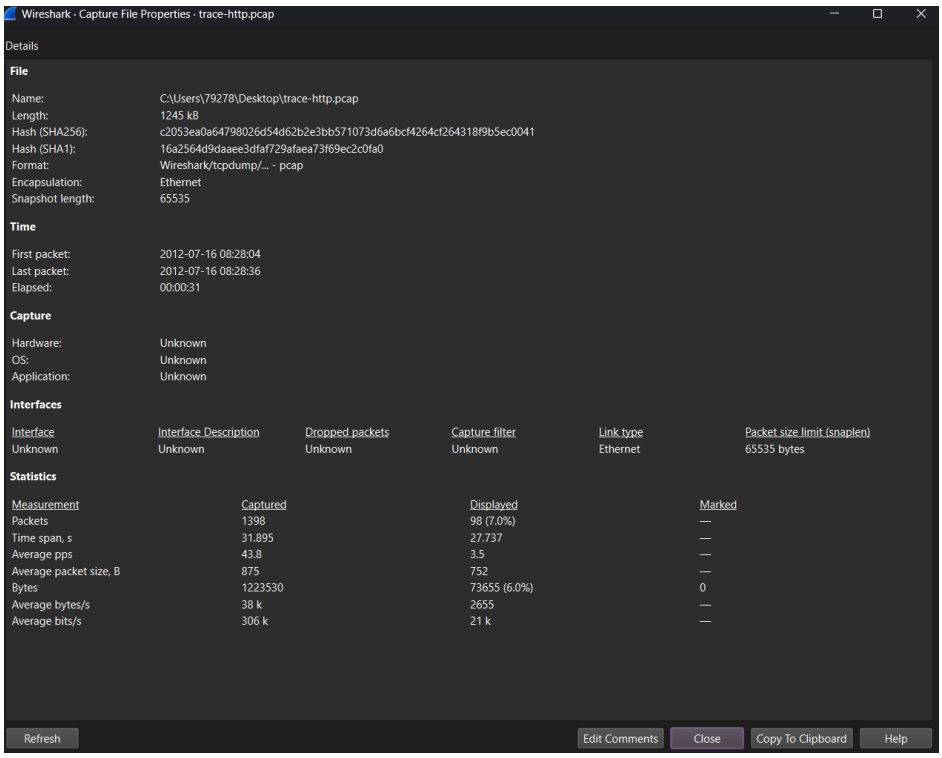


Рисунок 2 — Свойства файла захвата с применённым фильтром http.request (отображено 98 пакетов HTTP-запросов)

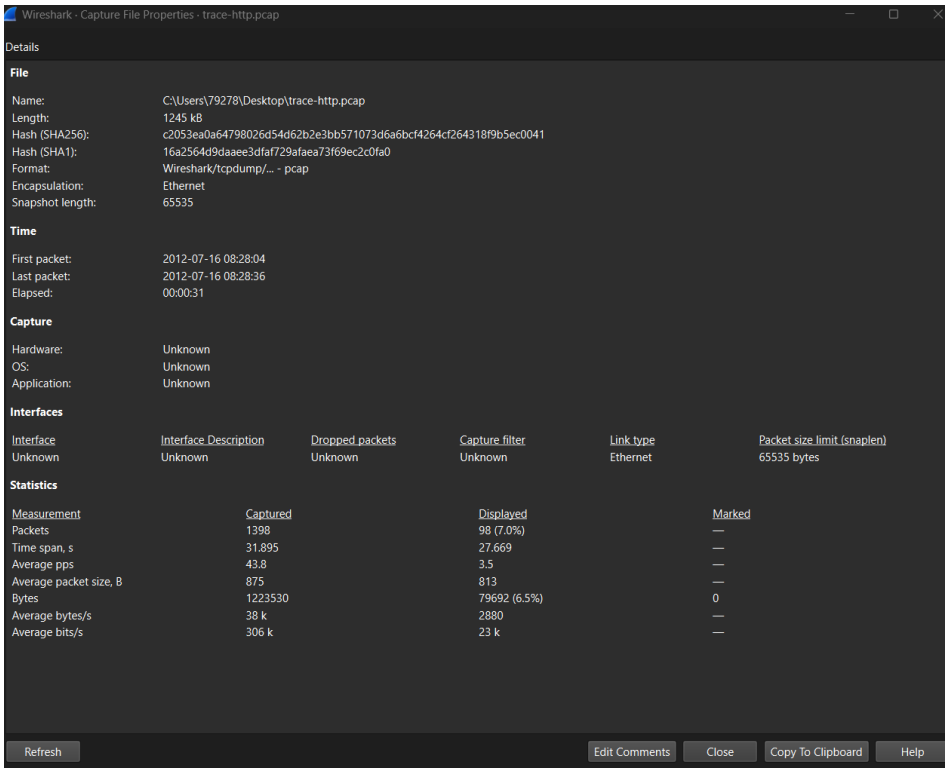


Рисунок 3 — Свойства файла захвата с применённым фильтром http.response (отображено 98 пакетов HTTP-ответов)

Задание №2. Изучение трассировки запроса и ответа.

2.1 Применение фильтра HTTP

Для отображения только HTTP-трафика был применён фильтр отображения http. Данный фильтр скрывает промежуточные TCP-сегменты и показывает только HTTP-запросы и ответы.

Результат: отображено 196 пакетов (98 HTTP-запросов и 98 HTTP-ответов).

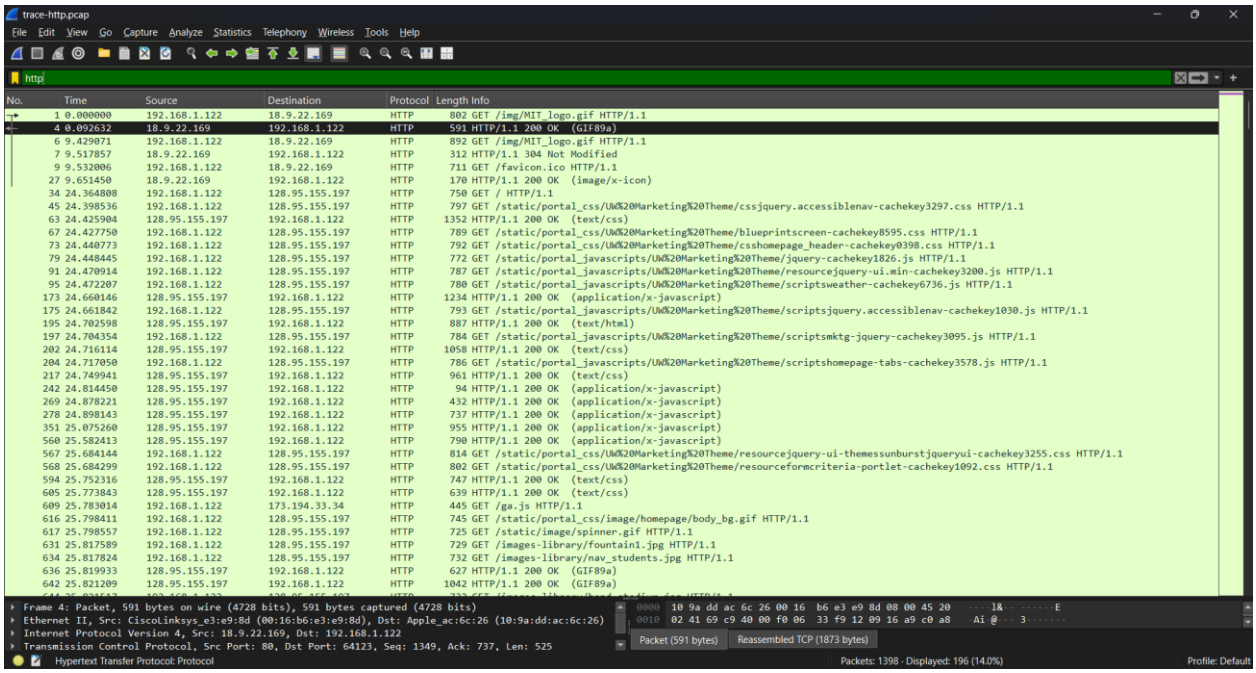


Рисунок 4 — Применение фильтра http (отображено 196 пакетов)

2.2 Фильтрация по протоколам

Фильтр tcp

При применении фильтра tcp отображается 1398 пакетов (100% от общего числа). Это является ожидаемым результатом, так как протокол HTTP работает на транспортном уровне поверх протокола TCP. Следовательно, весь HTTP-трафик в данном файле трассировки инкапсулирован в TCP-сегменты, и другие типы трафика (например, UDP) в этом файле отсутствуют.

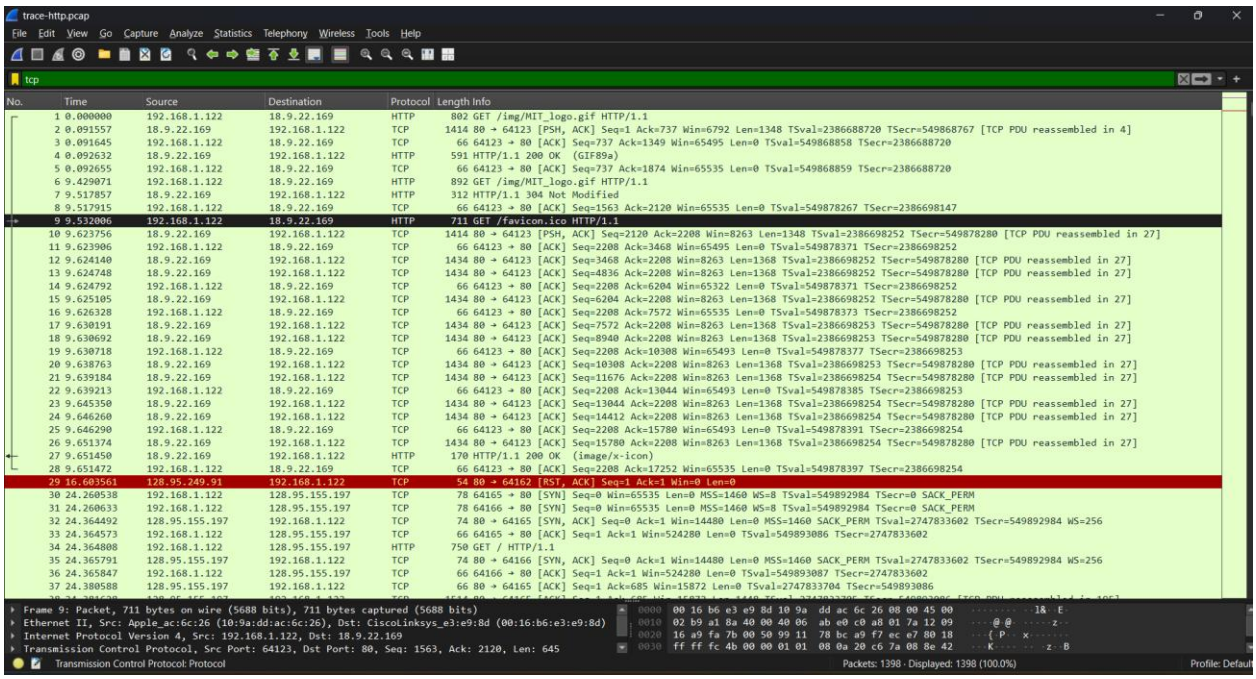


Рисунок 5 — Применение фильтра tcp (отображено 1398 пакетов)

Фильтр tcp.port == 80 || udp.port == 80

При применении фильтра tcp.port == 80 || udp.port == 80 также отображается 1398 пакетов (100%). Это объясняется тем, что файл trace-http.pcap содержит только HTTP-трафик, который по умолчанию использует порт 80. Поэтому все TCP-пакеты в файле имеют порт 80 в качестве источника или назначения.

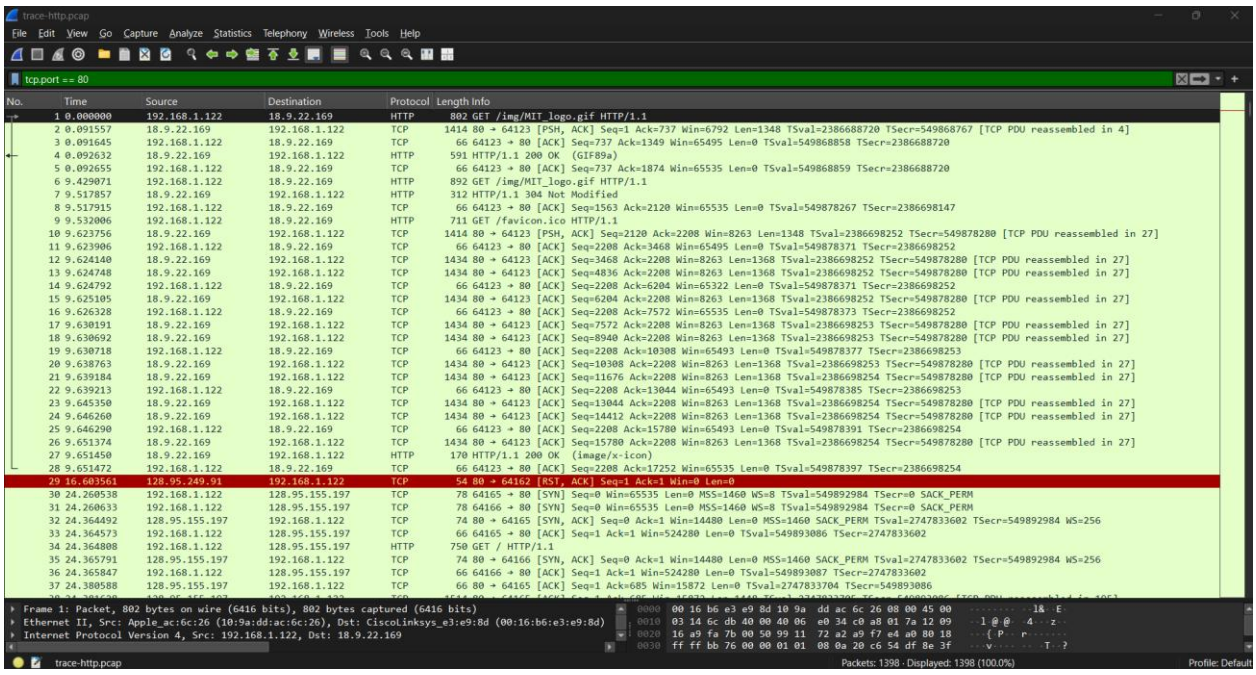


Рисунок 6 — Применение фильтра tcp.port == 80 || udp.port == 80 (отображено 1398 пакетов)

2.3 Изучение HTTP-заголовков GET-запроса

Первый GET-запрос в трассировке направлен на получение изображения логотипа MIT: GET /img/MIT_logo.gif HTTP/1.1.

В заголовках запроса обнаружены следующие поля:

1. Host: www.mit.edu — обязательный заголовок HTTP/1.1, указывающий доменное имя сервера, к которому обращается клиент. Необходим, так как на одном IP-адресе может размещаться несколько виртуальных хостов.

2. User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_7_3) AppleWebKit/536.11 (KHTML, like Gecko) Chrome/20.0.1132.57 Safari/536.11 — информация о браузере клиента (Chrome 20.0.1132.57) и операционной системе (Mac OS X 10.7.3). Сервер может использовать эту информацию для адаптации контента.

3. Accept: text/html,application/xhtml+xml,application/xml;q=0.9,/;q=0.8 — форматы контента (MIME-типы), которые клиент готов принять.

4. Accept-Encoding: gzip,deflate,sdch — поддерживаемые алгоритмы сжатия данных, чтобы сервер мог уменьшить размер передаваемых данных.

5. Accept-Language: en-US,en;q=0.8 — предпочтительный язык контента (английский, США).

6. Cache-Control: max-age=0 — клиент запрашивает актуальную версию ресурса, не используя кэш (например, при обновлении страницы через F5).

7. Connection: keep-alive — запрос на поддержание постоянного TCP-соединения для передачи нескольких запросов без повторного установления связи.

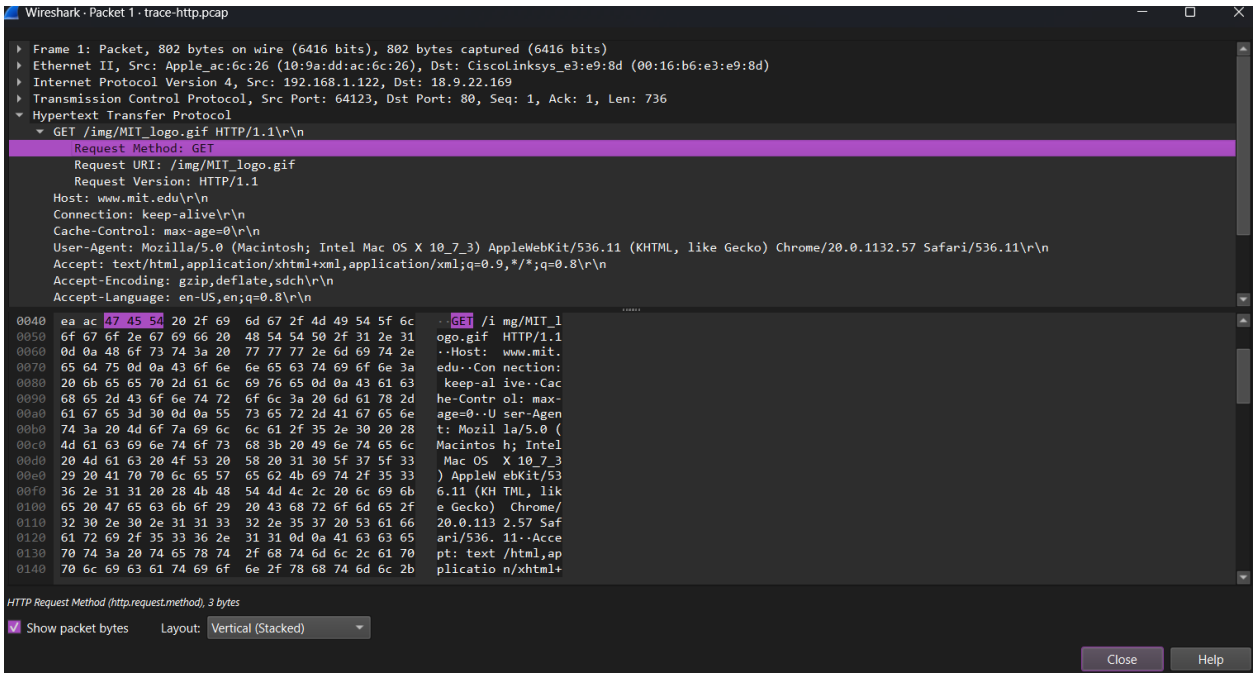


Рисунок 7 — Развернутый HTTP-блок первого GET-запроса с заголовками

2.4 Изучение HTTP-заголовков ответа

Ответ на первый GET-запрос: HTTP/1.1 200 OK. Код состояния 200 означает успешную передачу запрошенного ресурса.

В заголовках ответа обнаружены следующие поля:

1. Server: Apache/1.3.41 (Unix) mod_ssl/2.8.31 OpenSSL/0.9.8j — указывает на используемое серверное ПО: веб-сервер Apache версии 1.3.41, работающий на Unix, с модулем SSL и OpenSSL. Полезно для отладки, но часто скрывается из соображений безопасности.

2. Date: Mon, 16 Jul 2012 05:28:04 GMT — дата и время формирования ответа сервером (в формате GMT). Используется клиентом для синхронизации времени.

3. Last-Modified: Mon, 16 Jul 2012 04:00:09 GMT — время последнего изменения файла MIT_logo.gif на сервере. Используется для условных запросов: если клиент уже имеет кэшированную версию, он может отправить запрос If-Modified-Since и получить ответ 304 Not Modified вместо полной передачи файла.

4. Cache-Control: max-age=81125 — директива кэширования, указывающая, что ресурс можно хранить в кэше клиента в течение 81125 секунд (примерно 22,5 часа) без повторной проверки на сервере.

5. Expires: Tue, 17 Jul 2012 04:00:09 GMT — абсолютная дата, после которой кэшированная копия считается устаревшей.

6. ETag: "10e..." — уникальный хэш-идентификатор версии ресурса. Позволяет клиенту проверить, изменился ли файл, без загрузки всего содержимого (через заголовок If-None-Match). Если ETag совпадает, сервер возвращает 304 Not Modified.

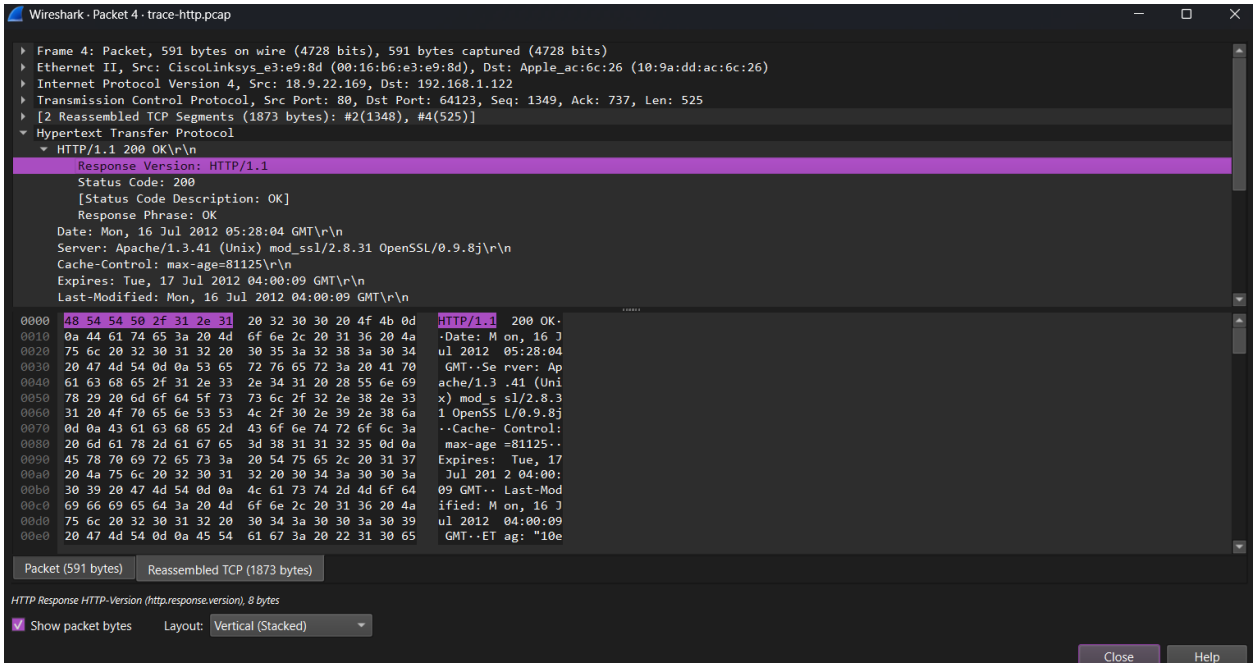


Рисунок 8 — Развернутый HTTP-блок ответа сервера (200 OK) на первый GET-запрос

2.5 Описание заголовков HTTP

Заголовки запроса

Host — обязательный заголовок HTTP/1.1. Указывает доменное имя и порт сервера, к которому направлен запрос. Необходим, так как на одном IP-адресе может размещаться несколько сайтов (виртуальный хостинг).

User-Agent — идентифицирует браузер, его версию и операционную систему клиента. Сервер может использовать эту информацию для адаптации

контента под возможности клиента (например, отдавать мобильную версию сайта).

Accept — сообщает серверу, какие MIME-типы контента клиент готов принять (text/html, image/gif и т.д.). Параметр q=0.9 указывает приоритет формата.

Accept-Encoding — указывает поддерживаемые алгоритмы сжатия (gzip, deflate, sdch), чтобы сервер мог уменьшить размер передаваемых данных и ускорить загрузку.

Accept-Language — предпочтительный язык контента. Сервер может использовать эту информацию для локализации.

Cache-Control: max-age=0 — означает, что клиент хочет получить свежую версию ресурса, не используя кэш. Обычно отправляется при принудительном обновлении страницы.

Connection: keep-alive — запрос на поддержание постоянного TCP-соединения для передачи нескольких запросов без повторного установления связи, что снижает задержки.

Cookie — хранит данные сессии и настройки, ранее сохранённые сервером. Позволяет серверу идентифицировать пользователя между запросами.

Заголовки ответа

Server — указывает программное обеспечение сервера (например, Apache/2.2.22). Полезно для отладки, но может быть скрыт из соображений безопасности, чтобы не раскрывать информацию о потенциальных уязвимостях.

Date — дата и время, когда сервер сформировал ответ (в формате GMT). Используется клиентом для синхронизации времени и проверки актуальности кэша.

Last-Modified — дата последнего изменения запрашиваемого файла на сервере. Используется для условных запросов: клиент может отправить запрос

If-Modified-Since и получить ответ 304 Not Modified вместо полной передачи файла, если файл не изменялся.

Cache-Control — директивы кэширования. Например, max-age=31536000 означает, что ресурс можно хранить в кэше 1 год без повторной проверки на сервере.

Expires — абсолютная дата, после которой кэшированная копия считается устаревшей. Устаревший заголовок, заменён на Cache-Control в HTTP/1.1.

ETag — уникальный хэш-идентификатор версии ресурса. Позволяет клиенту проверить, изменился ли файл, без загрузки всего содержимого (через заголовок If-None-Match). Если ETag совпадает, сервер возвращает 304 Not Modified, что экономит трафик.

Задание №3. Кэширование контента.

3.1 Повторный GET-запрос с заголовком If-Modified-Since

Второй блок записей в трассировке представляет собой повторную выборку по тому же URL (/img/MIT_logo.gif). Этот блок демонстрирует работу механизма кэширования HTTP.

Для анализа был выбран третий HTTP-запрос в трассировке: GET /img/MIT_logo.gif HTTP/1.1.

В отличие от первого запроса, этот запрос содержит дополнительный заголовок If-Modified-Since, который позволяет серверу решить, нужно ли отправлять свежий контент.

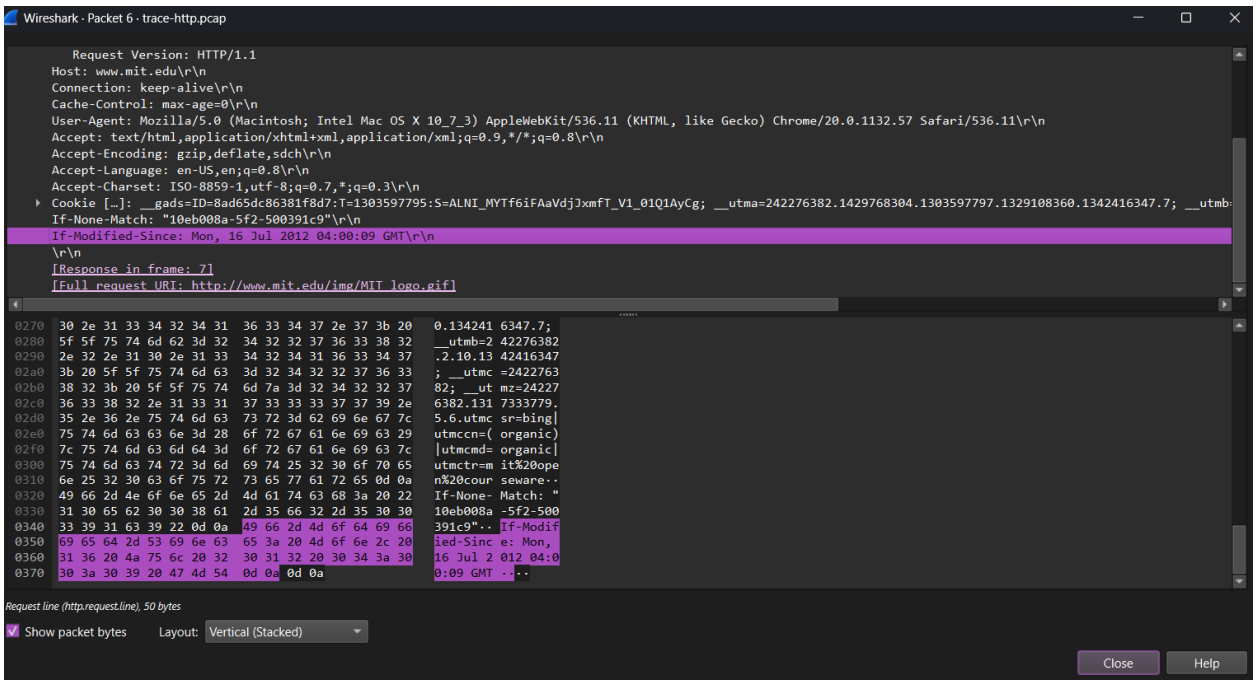


Рисунок 9 — HTTP-заголовок If-Modified-Since в повторном GET-запросе

Заголовок If-Modified-Since: Mon, 16 Jul 2012 04:00:09 GMT

Этот заголовок содержит временную метку из предыдущей загрузки контента, которую браузер сохранил в кэше. Сервер сравнивает эту дату с датой последнего изменения файла на сервере (заголовок Last-Modified). Если файл не изменялся после указанной даты, сервер не отправляет содержимое, а возвращает код 304 Not Modified.

В первом GET-запросе этот заголовок отсутствовал, так как кэш браузера был очищен и у него не было предыдущей загрузки контента.

3.2 Ответ сервера 304 Not Modified

Ответ на повторный GET-запрос: HTTP/1.1 304 Not Modified.

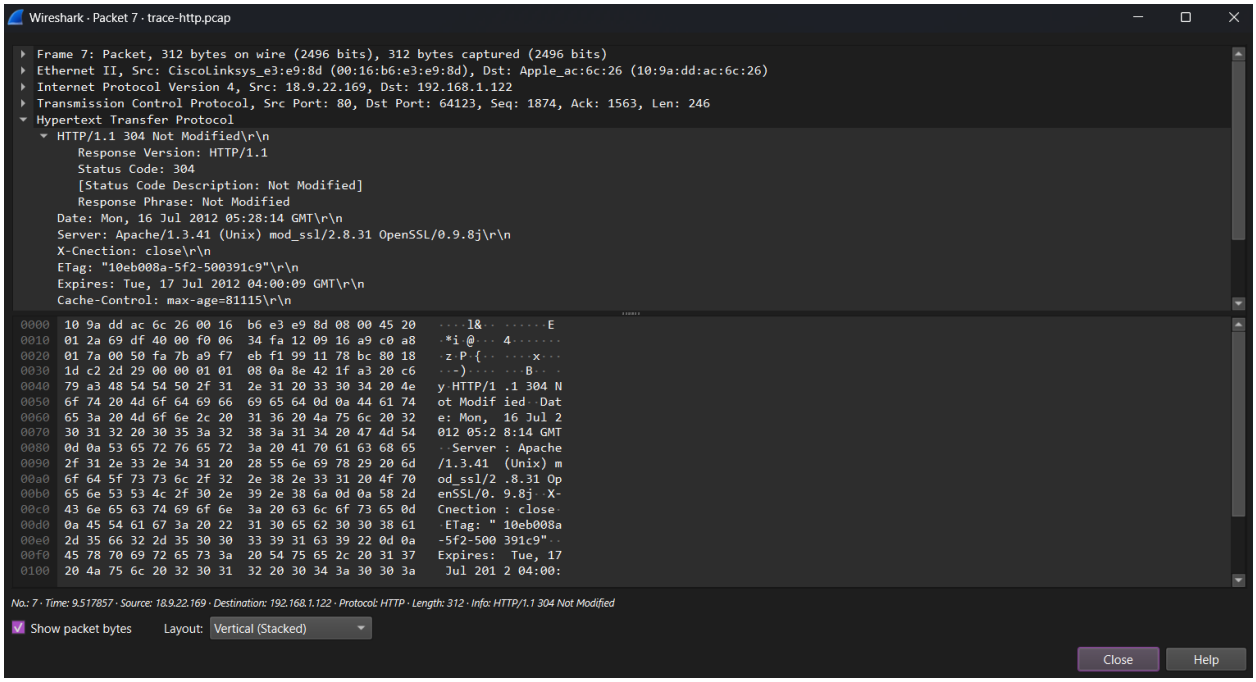


Рисунок 10 — HTTP-ответ 304 Not Modified на повторный GET-запрос

Код состояния 304 сообщает браузеру, что контент не изменился по сравнению с предыдущей копией, и кэшированный контент может быть использован для отображения. Ответ не содержит тело (контент), что экономит трафик.

Временная метка в заголовке If-Modified-Since совпадает со значением заголовка Last-Modified из предыдущего ответа сервера (Mon, 16 Jul 2012 04:00:09 GMT). Это временная метка сервера для последнего изменения контента, а не временная метка браузера или время загрузки.

3.3. Описание заголовка повторного GET-запроса.

При анализе третьего HTTP-запроса в трассировке были выявлены следующие заголовки:

Request Method: GET — метод запроса, означающий получение ресурса с сервера. Клиент запрашивает файл без передачи каких-либо данных на сервер.

Request URI: /img/MIT_logo.gif — путь к запрашиваемому файлу на сервере. В данном случае браузер обращается к изображению логотипа MIT, расположенному в директории /img/.

Request Version: HTTP/1.1 — версия протокола HTTP, используемая для обмена данными. Версия 1.1 поддерживает постоянные соединения и дополнительные заголовки кэширования.

Host: www.mit.edu — обязательный заголовок HTTP/1.1, указывающий доменное имя сервера, к которому направлен запрос. Необходим для корректной маршрутизации при виртуальном хостинге.

Connection: keep-alive — запрос на поддержание постоянного TCP-соединения между клиентом и сервером. Это позволяет передавать несколько запросов без повторного установления соединения, что снижает задержки.

Cache-Control: max-age=0 — директива кэширования, указывающая, что клиент запрашивает проверку актуальности ресурса. Значение max-age=0 означает, что браузер не будет использовать кэш напрямую, а сначала проверит, не изменился ли файл на сервере.

User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_7_3) AppleWebKit/536.11 (KHTML, like Gecko) Chrome/20.0.1132.57 Safari/536.11 — заголовок, содержащий информацию о браузере клиента (Chrome версии 20.0.1132.57) и операционной системе (Mac OS X 10.7.3). Сервер может использовать эти данные для адаптации контента.

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,/;q=0.8 — список MIME-типов контента, которые клиент готов принять. Параметр q указывает приоритет формата: чем выше значение, тем предпочтительнее тип. Accept-Encoding: gzip,deflate,sdch — перечисление поддерживаемых клиентом алгоритмов сжатия данных. Сервер может использовать один из этих методов для уменьшения размера передаваемого контента.

Accept-Language: en-US,en;q=0.8 — предпочтительный язык контента. В данном случае клиент предпочитает английский язык (США), а также принимает английский в общем.

If-Modified-Since: Mon, 16 Jul 2012 04:00:09 GMT — ключевой заголовок для механизма кэширования. Содержит дату последнего известного изменения файла, которую браузер сохранил из предыдущего ответа сервера (заголовок Last-Modified). Сервер сравнивает эту дату с текущей датой изменения файла и возвращает контент только если файл был изменён после указанной даты. В противном случае сервер возвращает код 304 Not Modified.

If-None-Match: "10e0bba-5f2-10409559" — заголовок условного запроса, содержащий значение ETag предыдущей версии файла. ETag — это уникальный хэш-идентификатор версии ресурса. Сервер сравнивает переданный ETag с текущим, и если они совпадают, возвращает код 304 Not Modified без передачи содержимого файла. Этот механизм более точный, чем If-Modified-Since, так как позволяет отслеживать изменения с точностью до байта.

В первом GET-запросе заголовки If-Modified-Since и If-None-Match отсутствовали, так как кэш браузера был очищен и у него не было предыдущей загрузки контента для сравнения.

3.4 Запрашиваемое изображение

Запрос был направлен на получение изображения логотипа MIT в формате GIF по адресу /img/MIT_logo.gif.



Рисунок 11 — GIF-изображение логотипа MIT (/img/MIT_logo.gif), по которому обрабатывался HTTP-запрос

3.5 Механизм кэширования HTTP

Механизм кэширования HTTP работает следующим образом:

1. Первый запрос: Браузер отправляет GET-запрос без заголовков If-Modified-Since и If-None-Match, так как кэш пуст. Сервер возвращает полный контент с заголовками Last-Modified и ETag.

2. Кэширование: Браузер сохраняет контент в локальном кэше вместе с метаданными (дата последнего изменения и ETag).

3. Повторный запрос: При повторном обращении к тому же ресурсу браузер добавляет заголовки:

- If-Modified-Since — дата из Last-Modified предыдущего ответа

- If-None-Match — значение ETag предыдущего ответа

4. Проверка на сервере: Сервер сравнивает эти значения с текущими. Если файл не изменялся, сервер возвращает код 304 Not Modified без тела ответа.

5. Использование кэша: Браузер получает код 304 и использует локальную кэшированную копию для отображения.

Задание №4. Сложные страницы.

4.1 Анализ запроса сложной веб-страницы

Четвертая выборка в трассировке представляет собой запрос более сложной веб-страницы, которая содержит множество встроенных ресурсов. В отличие от простого запроса изображения, при загрузке такой страницы браузер выполняет следующие действия:

1. Загружает исходный HTML-документ

2. Анализирует HTML и обнаруживает встроенные ресурсы (CSS, JavaScript, изображения, шрифты)

3. Загружает все встроенные ресурсы, необходимые для корректного отображения страницы

4. Выполняет JavaScript-сценарии, которые могут запрашивать дополнительные ресурсы

В результате одна страница может включать множество GET-запросов к различным серверам.

Для анализа был выбран запрос GET / HTTP/1.1 — запрос к корневой странице сайта.

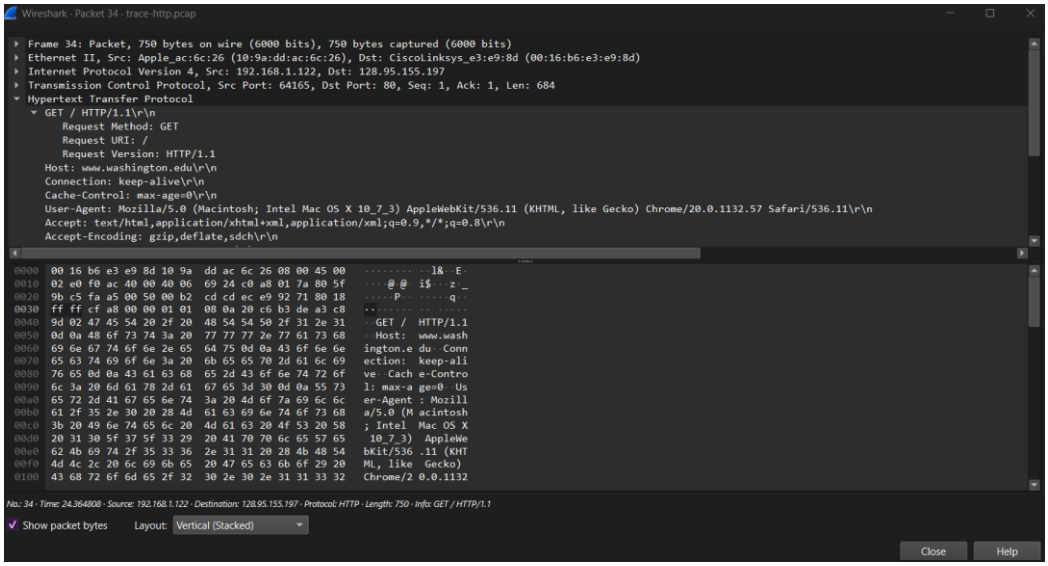


Рисунок 12 — GET-запрос основной веб-страницы в трассировке

4.2 Распределение нагрузки по серверам (HTTP Load Distribution)

Для обобщения GET-запросов была вызвана панель HTTP Load Distribution через меню Статистика → HTTP → Load Distribution.

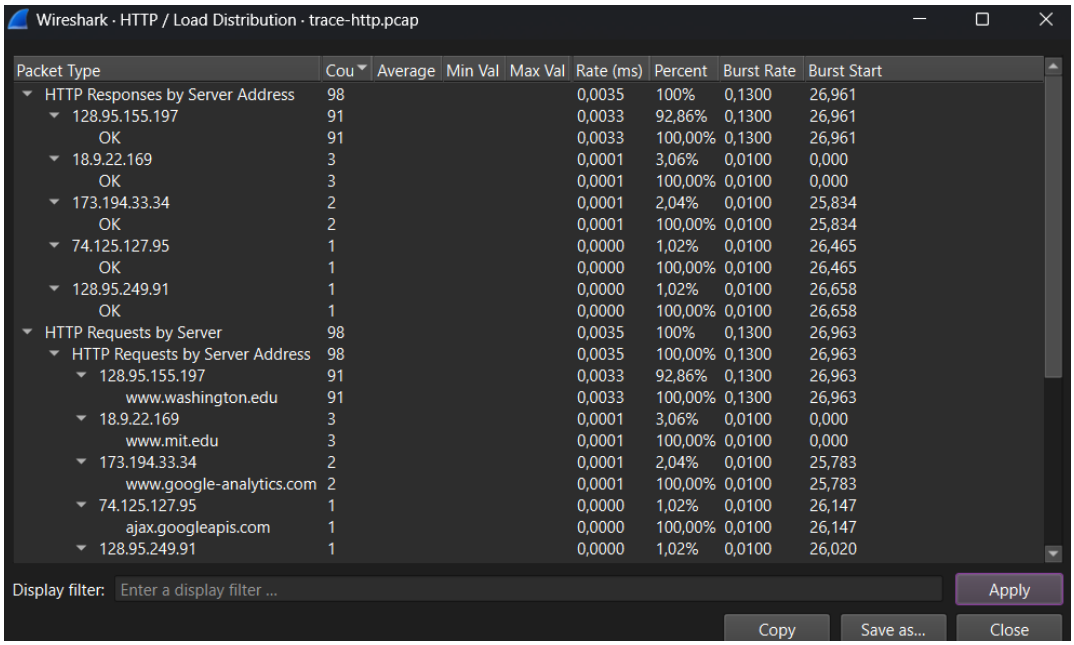


Рисунок 13 — Распределение HTTP-запросов по серверам (HTTP Load Distribution)

На панели показана информация о том, сколько запросов было сделано к различным серверам. Согласно полученным данным:

1. Всего HTTP-запросов: 98
2. Количество различных серверов: 5

Распределение запросов по серверам:

Сервер	Количество запросов	Процент
128.95.155.197 (основной сервер)	91	92.86%
18.9.22.169 (www.mit.edu)	3	3.06%
173.194.33.34 (google-analytics.com)	2	2.04%
74.125.127.95 (ajax.googleapis.com)	1	1.02%
Другие серверы	1	1.02%

Этот результат демонстрирует важный аспект современного веб-серфинга: при загрузке одной страницы браузер запрашивает контент с нескольких серверов. Эти серверы могут включать:

1. Основной сервер сайта — хранит HTML, CSS, основные изображения
2. CDN (Content Delivery Networks) — сети доставки контента для ускорения загрузки
3. Аналитические сервисы (google-analytics.com) — отслеживание посещаемости и поведения пользователей
4. Библиотеки и фреймворки (ajax.googleapis.com) — внешние JavaScript-библиотеки
5. Рекламные сети — показ рекламного контента

При обычном серфинге пользователь может не подозревать, что страница загружает данные со множества сторонних серверов, что имеет последствия для конфиденциальности, безопасности и скорости загрузки.

4.3 Счетчик HTTP-пакетов (HTTP Packet Counter)

Для получения детальной сводки по типам HTTP-запросов и ответов была открыта панель HTTP Packet Counter через меню Статистика → HTTP → HTTP Packet Counter.

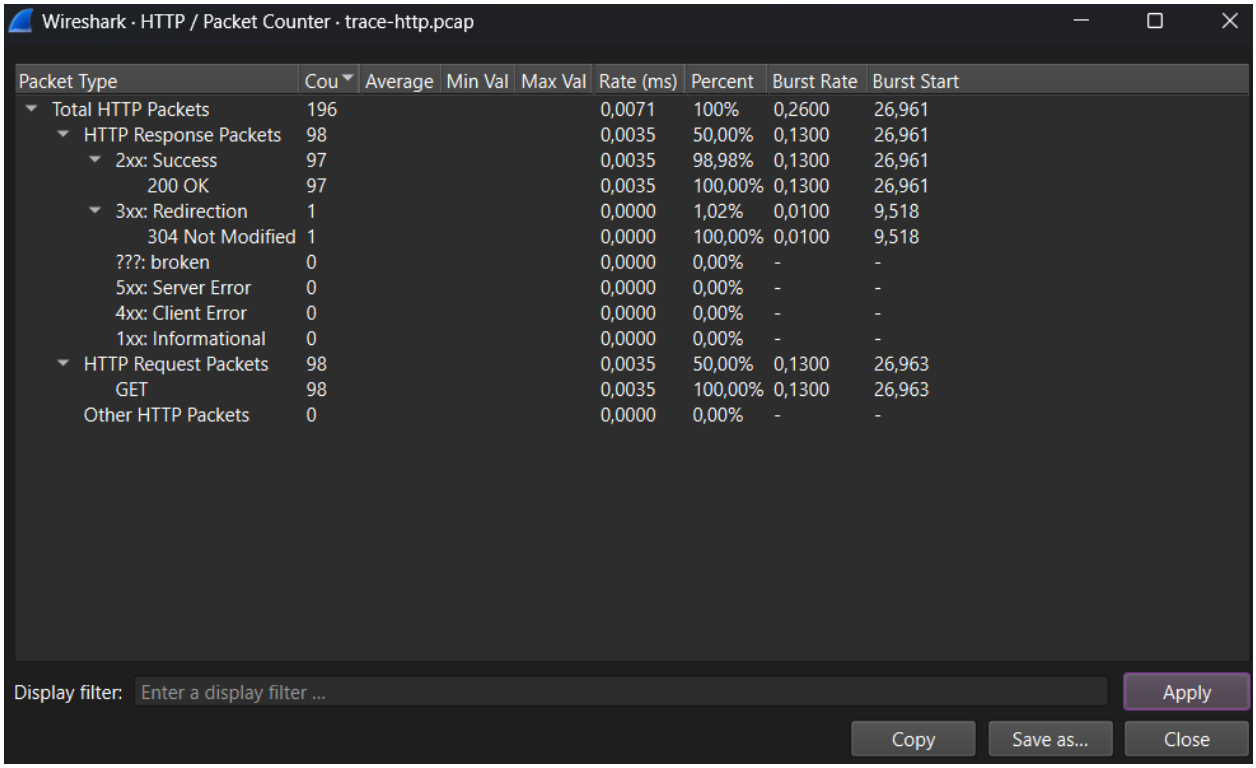


Рисунок 14 — Счетчик HTTP-пакетов по типам запросов и ответов

На панели отображается следующая статистика:

Общая статистика:

1. Всего HTTP-пакетов (Total HTTP Packets): 196
2. HTTP Request Packets (запросы): 98
3. HTTP Response Packets (ответы): 98

Типы запросов: GET: 98 — все запросы были типа GET (получение ресурсов с сервера)

Типы ответов:

2xx: Success (Успешные ответы) — 97 пакетов (98,98%)

200 OK: 97 — успешная передача запрошенного ресурса. Сервер нашел файл и отправил его клиенту.

3xx: Redirection (Перенаправления) — 1 пакет (1,02%)

304 Not Modified: 1 — ресурс не изменился с момента последней загрузки. Браузер использует кэшированную версию, что экономит трафик и время.

4xx: Client Error (Ошибки клиента) — 0 пакетов

Ошибки клиента (404 Not Found, 403 Forbidden и т.д.) отсутствуют.

5xx: Server Error (Ошибки сервера) — 0 пакетов

Ошибки сервера (500 Internal Server Error и т.д.) отсутствуют.

??? Broken (Поврежденные пакеты) — 0 пакетов

Такая статистика показывает, что большинство запросов завершились успешно (200 OK), и лишь один ресурс был взят из кэша (304 Not Modified). Отсутствие ошибок 4xx и 5xx свидетельствует о корректной работе серверов и правильности запросов клиента.

4.4 Анализ типов загружаемого контента

В трассировке были запрошены различные типы ресурсов, необходимые для полноценного отображения веб-страницы:

1. HTML-документы — основная структура страницы
2. CSS-таблицы стилей (.css) — оформление и визуальное представление
3. JavaScript-скрипты (.js) — интерактивность и динамическое поведение
4. Изображения (.gif, .png, .jpg, .ico) — графические элементы и иконки
5. Шрифты — кастомные шрифты для типографики

Каждый из этих ресурсов загружается отдельным HTTP-запросом. В совокупности они формируют полную веб-страницу, которую видит пользователь в браузере.

4.5 Водопадная диаграмма загрузки страницы

Для более подробного анализа процесса загрузки страницы был использован онлайн-сервис WebPageTest.org, который проводит тестирование производительности веб-страниц и генерирует детальные отчеты.

Тестирование проводилось для сайта https://www.mit.edu (аналогично трассировке).

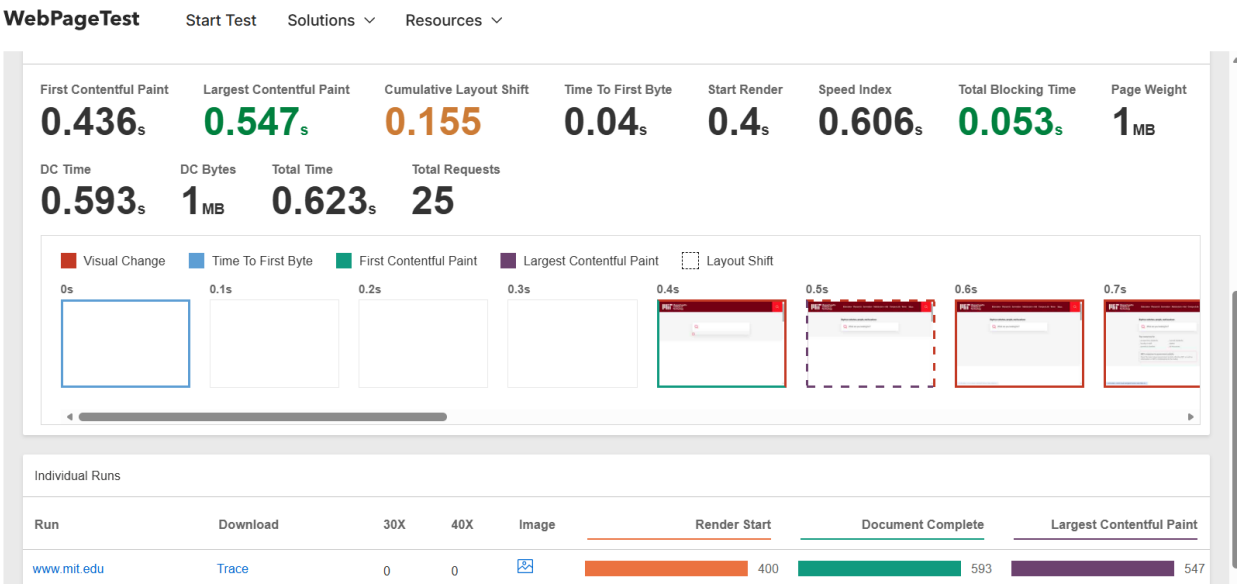


Рисунок 15 — Водопадная диаграмма загрузки страницы (результат тестирования WebPageTest)

На водопадной диаграмме наглядно показан процесс загрузки каждого ресурса:

Этапы загрузки:

1. DNS Lookup — разрешение доменного имени в IP-адрес
2. Initial Connection — установление TCP-соединения с сервером
3. SSL/TLS Handshake — установление безопасного HTTPS-соединения
4. Time to First Byte (TTFB) — время до получения первого байта ответа от сервера
5. Content Download — загрузка содержимого ресурса

Наблюдения:

Последовательность загрузки:

1. Сначала загружается основной HTML-документ
2. Затем браузер обнаруживает встроенные ресурсы и начинает их загружать
3. CSS и JavaScript загружаются параллельно (несколько соединений одновременно)
4. Изображения загружаются после критических ресурсов

Параллельная загрузка:

1. Современные браузеры открывают несколько одновременных соединений к серверу (обычно 6-8)
2. Это позволяет загружать ресурсы параллельно, ускоряя общую загрузку страницы

Блокирующие ресурсы:

1. CSS-файлы блокируют отображение страницы до полной загрузки
2. JavaScript-файлы могут блокировать выполнение других скриптов
3. Критические ресурсы загружаются в первую очередь

4.6 Ценность информации для веб-разработки

Анализ загрузки страницы с помощью инструментов трассировки (Wireshark) и специализированных сервисов производительности (PageSpeed, WebPageTest) имеет огромную ценность для веб-разработки по нескольким причинам:

1. Оптимизация производительности

Выявление узких мест:

- Водопадная диаграмма показывает, какие ресурсы загружаются дольше всего
- Можно определить блокирующие рендеринг CSS и JavaScript файлы
- Видно количество одновременных соединений к серверу

Рекомендации по оптимизации:

- Минификация CSS и JavaScript (удаление пробелов, комментариев, сокращение имен переменных)
- Сжатие ресурсов с помощью gzip/deflate для уменьшения размера передаваемых данных
- Объединение мелких файлов для уменьшения количества HTTP-запросов
- Отложенная загрузка (lazy loading) для некритичных изображений и контента
- Асинхронная загрузка JavaScript (атрибуты async и defer) для неблокирующей загрузки

2. Улучшение пользовательского опыта (UX)

Скорость загрузки:

- Каждая дополнительная секунда загрузки увеличивает процент отказов на 7%
- Быстрая загрузка повышает конверсию и удержание пользователей
- 53% пользователей покидают сайт, если он загружается дольше 3 секунд

Визуальная стабильность:

- Анализ показывает, когда страница становится интерактивной
- Можно оптимизировать порядок загрузки критических ресурсов для быстрого отображения контента

3. Оптимизация использования кэша

Анализ заголовков кэширования:

- Проверка корректности заголовков Cache-Control, Expires, ETag
- Определение ресурсов, которые можно кэшировать на длительный срок (статичные изображения, CSS, JS)
- Выявление ресурсов, которые загружаются повторно без необходимости

Результат:

- Уменьшение количества запросов к серверу при повторных посещениях
- Снижение нагрузки на сервер и сетевого трафика
- Ускорение загрузки для постоянных пользователей

4. Безопасность и конфиденциальность

Выявление сторонних ресурсов:

- Анализ показывает, к каким сторонним серверам обращается страница
- Можно обнаружить нежелательные трекеры и скрипты аналитики
- Оценка рисков утечки данных через сторонние ресурсы

HTTPS и безопасность:

- Проверка использования безопасного соединения для всех ресурсов
- Выявление смешанного контента (HTTP + HTTPS), который может быть уязвимостью

5. Отладка и диагностика

Выявление ошибок:

- HTTP-ошибки 404 (ресурс не найден) — битые ссылки

- HTTP-ошибки 500 (ошибки сервера) — проблемы на стороне сервера
- Таймауты и прерванные соединения — проблемы с сетью или сервером

Анализ зависимостей:

- Понимание, какие ресурсы от каких зависят
- Выявление циклических зависимостей
- Оптимизация порядка загрузки

6. SEO-оптимизация

Факторы ранжирования:

- Скорость загрузки — прямой фактор ранжирования Google
- Мобильная производительность (Mobile-First Indexing)
- Core Web Vitals (LCP, FID, CLS) — метрики качества страницы, влияющие на позиции в поиске

Инструменты:

- Google PageSpeed Insights дает конкретные рекомендации по улучшению
- WebPageTest предоставляет детальные метрики производительности
- Lighthouse проводит комплексный аудит доступности, SEO и лучших практик

7. Экономия ресурсов

Для владельца сайта:

- Снижение нагрузки на сервер
- Уменьшение расходов на хостинг и трафик
- Увеличение пропускной способности для большего числа пользователей

Для пользователя:

- Экономия мобильного трафика (особенно важно при лимитированных тарифах)
- Снижение потребления батареи на мобильных устройствах
- Быстрая загрузка даже при медленном соединении (3G, EDGE)

Выводы по Заданию 4

1. Сложные веб-страницы состоят из множества ресурсов (HTML, CSS, JavaScript, изображения, шрифты), которые загружаются с разных

серверов. Один запрос к странице может генерировать десятки дополнительных запросов.

2. HTTP Load Distribution показал, что при загрузке одной страницы было сделано 98 запросов к 5 различным серверам, включая основные серверы сайта, CDN, аналитические сервисы (Google Analytics) и библиотеки (Google APIs).

3. HTTP Packet Counter продемонстрировал, что из 196 HTTP-пакетов:

- 98 запросов типа GET
- 97 успешных ответов 200 OK (98,98%)
- 1 ответ 304 Not Modified (1,02%) — использование кэша
- Ошибки клиента и сервера отсутствуют

4. Водопадная диаграмма наглядно показала последовательность и время загрузки каждого ресурса, выявив блокирующие рендеринг ресурсы и возможность параллельной загрузки.

5. Анализ производительности с помощью специализированных сервисов (PageSpeed, WebPageTest) предоставляет ценную информацию для оптимизации веб-страниц, что напрямую влияет на пользовательский опыт, SEO-позиции и конверсию.

6. Понимание процесса загрузки позволяет веб-разработчикам принимать обоснованные решения об оптимизации, что приводит к более быстрым, надежным и эффективным веб-приложениям.